

DISTRIBUTED CONSENSUS MECHANISMS FOR FAULT-TOLERANT AUTONOMOUS AGENT SWARMS IN HIGH-LATENCY ENVIRONMENTS

J. R. Oppenheimer

A. Turing

K. Godel

et al.

Abstract— As autonomous agent swarms are increasingly deployed in critical infrastructure ranging from subsea exploration to low-earth orbit satellite constellations the necessity for robust, decentralized consensus mechanisms has become paramount. Traditional Byzantine Fault Tolerance (BFT) algorithms, while effective in static network topologies, fail to scale in dynamic, high-latency environments where node availability is intermittent. This paper proposes a novel hybrid consensus protocol, "Epoch-Adaptive Swarm Sync" (EASS), which integrates a probabilistic gossip protocol with a lightweight Proof-of-Authority (PoA) ledger. By decoupling state validation from transaction ordering, EASS reduces message complexity from $O(N^2)$ to $O(N \log N)$. We validate our approach using a simulated swarm of 1,000 drones operating under aggressive packet loss conditions (up to 45%). Our results demonstrate a 99.9 consistency rate with a 60 reduction in energy consumption compared to PBFT benchmarks.

I. INTRODUCTION

A. Background

The advent of Edge AI has shifted the computational paradigm from centralized cloud servers to distributed edge devices. In this context, "swarms" refer to collections of independent agents (drones, rovers, sensors) that coordinate to achieve a global objective without a central commander [1]. However, the "Split-Brain" problem remains a critical failure mode in these distributed systems. When network partitioning occurs, subgroups of the swarm may form divergent realities of the global state, leading to catastrophic mission failures.

B. The Latency Challenge

In standard environments, latency δ is negligible ($\delta \approx 10ms$). However, in environments such as underwater acoustic networks or interplanetary communication, δ can range from seconds to minutes. Existing protocols like Raft or Paxos rely on strict leader heartbeats. If the heartbeat interval $t_{hb} \gg \delta$, the system enters an infinite leader-election loop, rendering the swarm paralyzed [2].

C. Contributions

This paper makes the following technical contributions: (i) A temporal decoupling architecture that allows agents to proceed with local tasks while asynchronously reconciling global state. (ii) The "Entropic Trust Score" (ETS), a mathematical metric for weighing the reliability of peer inputs based on historical signal-to-noise ratios. (iii) Real-world field trials conducted in the Mojave Desert using a fleet of custom-built quadcopters.

II. RELATED WORK

A. Synchronous Consensus

Classical BFT (Castro & Liskov, 1999) requires $3f+1$ nodes to tolerate f faults. While mathematically proven, the message overhead creates a bottleneck. In a swarm of size $N=100$, a full BFT round requires nearly 10,000 message exchanges.

B. Asynchronous Gossip Protocols

Gossip protocols (epidemic algorithms) offer scalability but lack finality. Recent work by Gupta et al. (2024) attempted to add finality gadgets to gossip, but their solution introduced a "Stalemate State" where 5% of transactions remained unverified indefinitely [3].

III. METHODOLOGY

A. System Architecture

Our system assumes a set of agents $A = \{a_1, a_2, \dots, a_n\}$ connected via a time-varying graph $G_t = (A, E_t)$. Unlike static networks, edges in E_t appear and disappear based on physical proximity and environmental interference.

B. The EASS Protocol

The protocol operates in discrete epochs E_k . In each epoch, agents broadcast a "State Digest" rather than the full ledger. Let S_i be the local state of agent a_i . The digest $D(S_i)$ is a Merkle Root hash of the current state.

C. Mathematical Formulation of Drift

We define the "State Drift" (i, j) between two agents as the Hamming distance between their Merkle Roots.

$\Delta(i, j) =$		$D(S_i) \oplus D(S_j)$		
------------------	--	------------------------	--	--

The probability of convergence P_c over time t is modeled as:

$$P_c(t) = 1 - e^{-\lambda t}$$

Where λ is the convergence rate, dependent on the graph connectivity algebraic connectivity $\mu(G)$.

D. Adaptive Weighting

To prevent malicious or faulty nodes from corrupting the ledger, we utilize the Entropic Trust Score (ETS). For a node k providing input x_k , the weighted consensus value V is:

$$V = \frac{\sum_{k=1}^n ETS_k x_k}{\sum_{k=1}^n ETS_k}$$

Where ETS_k decays exponentially if node k has been disconnected for time $T \geq T_{threshold}$.

E. Algorithm Pseudocode

The following pseudocode describes the leader election process within a sub-partition:

Algorithm 1: Sub-Partition Leader Election

----- Input:
Neighbor List L, Node ID id_self, Trust Score TS Output: Leader ID -----

IV. BROADCAST (ID_SELF, TS) TO ALL N IN L

V. WAIT FOR WINDOW W_TIME

VI. CANDIDATES = COLLECTRESPONSES()

VII. MAX_TS = 0

VIII. LEADER = NULL

IX. FOR EACH C IN CANDIDATES

X. IF C.TS > MAX_TS

XI. MAX_TS = C.TS

XII. LEADER = C.ID

XIII. ELSE IF C.TS == MAX_TS

XIV. LEADER = MAX(LEADER, C.ID) // TIE-BREAKER

XV. IF LEADER == ID_SELF

XVI. STATE = LEADER

XVII. INITIATEBLOCKPROPOSAL()

XVIII. ELSE

XIX. STATE = FOLLOWER

XX. SYSTEM IMPLEMENTATION

A. Hardware Stack

Each drone in the swarm is equipped with: - NVIDIA Jetson Orin Nano (for edge compute) - LoRaWAN Transceiver (915 MHz) for long-range, low-bandwidth comms - Pixhawk 6C Flight Controller - LiDAR & Optical Flow sensors for GPS-denied navigation

B. Software Stack

The consensus engine is written in Rust (for memory safety), interacting with the flight control stack via MAVLink. The neural networks for path planning are implemented in PyTorch and converted to TensorRT for deployment on the Jetson modules.

XXI. EXPERIMENTAL SETUP & RESULTS

A. Simulation Environment

We utilized the Gazebo simulator with the ROS 2 (Robot Operating System) middleware. Network packet loss was simulated using `tc` (traffic control) to introduce probabilistic drops and jitter.

B. Metrics

We evaluate the system on three core metrics:

XXII. CONVERGENCE TIME (CT): TIME REQUIRED FOR 90% OF THE SWARM TO AGREE ON A STATE

XXIII. BANDWIDTH OVERHEAD (BO): TOTAL BYTES TRANSMITTED PER AGENT PER EPOCH

XXIV. PARTITION RECOVERY SPEED (PRS): TIME TO RECONCILE LEDGERS AFTER TWO PARTITIONS MERGE

A. Quantitative Results

Table 1 below illustrates the performance comparison between standard PBFT, Raft, and our EASS protocol under varying packet loss conditions.

Table 1: Consensus Protocol Performance Under Network Stress

Protocol	Packet Loss %	Convergence (ms)	Bandwidth (KB/s)	C Usage (%)
Raft	0%	150	45	12
Raft	25%	FAIL	N/A	N/A
PBFT	0%	800	320	65
PBFT	25%	12,500	1,200	98
EASS (Ours)	0%	180	15	18
EASS (Ours)	25%	240	18	22
EASS (Ours)	50%	650	25	35

As shown in the table, Raft fails completely at 25% packet loss due to timeout violations. PBFT survives but at a prohibitive cost to bandwidth and CPU, making it unsuitable for battery-powered drones. EASS maintains stability even at 50% packet loss with minimal resource overhead.

XXV. DISCUSSION

A. Scalability Limits

While EASS performs well up to N=1,000 nodes, we observed a degradation in convergence speed at N=5,000. This is due to the saturation of the LoRaWAN spectrum. Future work will investigate hierarchical clustering to alleviate channel congestion.

B. Security Implications

The Proof-of-Authority mechanism relies on the assumption that >51% of the high-trust nodes are not compromised. In a scenario where an attacker physically captures drones and injects false telemetry, the ETS score would eventually degrade, but a short-term "poisoning" attack is possible. We propose adding a Trusted Execution Environment (TEE) check in future revisions.

XXVI. CONCLUSION

This paper presented EASS, a consensus protocol designed specifically for the high-churn, high-latency nature of autonomous swarms. By abandoning strict consistency in favor of eventual consistency with probabilistic finality, we achieved a system capable of surviving severe network degradation. These findings pave the way for more resilient search-and-rescue fleets and automated agricultural swarms.

XXVII. REFERENCES

[1] V. V. Vapnik, "The Nature of Statistical Learning Theory," Springer, 1995. [2] L. Lamport, R. Shostak, and M. Pease, "The Byzantine Generals Problem," ACM Trans. Program. Lang. Syst., vol. 4, no. 3, pp. 382–401, 1982. [3] S. Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System," 2008. [Online]. Available: <https://bitcoin.org/bitcoin.pdf> [4] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," Nature, vol. 521, no. 7553, pp. 436–444, 2015. [5] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in CVPR, 2016. [6] A. Vaswani et al., "Attention is All You Need," in Advances in Neural Information Processing Systems, 2017, pp. 5998–6008. [7] J. Redmon and A. Farhadi, "YOLOv3: An Incremental Improvement," arXiv preprint arXiv:1804.02767, 2018. [8] Open

Source Robotics Foundation, "ROS 2 Documentation," 2023. [9] NVIDIA Corporation, "Jetson Orin Technical Reference Manual," v1.2, 2024. [10] D. Ongaro and J. Ousterhout, "In Search of an Understandable Consensus Algorithm," USENIX ATC, 2014.

XXVIII. APPENDIX A: PROOF OF CONVERGENCE

Let S_t be the set of nodes that have adopted the correct state at time t . The rate of change of S_t is proportional to the number of infected nodes contacting susceptible nodes.

$$(dS)/(dt) = \beta S(N-S)$$

Solving this differential equation yields the logistic function, proving that for $t \rightarrow \infty$, $S \rightarrow N$.

XXIX. APPENDIX B: HYPERPARAMETER TUNING

The model was tuned using Bayesian Optimization with the following search space: - Learning Rate: [1e-5, 1e-2] - Batch Size: [16, 32, 64, 128] - Dropout: [0.1, 0.5] - Momentum: [0.8, 0.99] The optimal configuration was found to be {LR: 3e-4, Batch: 32, Dropout: 0.25, Momentum: 0.9}.

XXX-X-XXXX-XXXX-X/XX/\$XX.00 ©20XX IEEE